



Using large language models for template detection from security event logs

Risto Vaarandi¹ · Hayretin Bahşi^{1,2}

© The Author(s) 2025

Abstract

In modern IT systems and computer networks, real-time and offline event log analysis is a crucial part of cyber security monitoring. In particular, event log analysis techniques are essential for the timely detection of cyber attacks and for assisting security experts with the analysis of past security incidents. The detection of line patterns or templates from unstructured textual event logs has been identified as an important task of event log analysis since detected templates represent event types in the event log and prepare the logs for downstream online or offline security monitoring tasks. During the last 2 decades, a number of template mining algorithms have been proposed. However, many proposed algorithms rely on traditional data mining techniques, and the usage of Large Language Models (LLMs) has received less attention so far. Also, most approaches that harness LLMs are supervised, and unsupervised LLM-based template mining remains an understudied area. The current paper addresses this research gap and investigates the application of LLMs for unsupervised detection of templates from unstructured security event logs.

Keywords LLM-based template detection from security event logs · Template detection from event logs · Security event log analysis · LLMs for event log analysis · LLMs for cyber security

1 Introduction

Event log analysis is an important cyber security monitoring technique. For example, modern Security Operations Center (SOC) platforms like Splunk [1] and ElasticStack [2] rely on collecting and analyzing event logs from a large number of sources, whereas dedicated event correlation tools like SEC [3] have been designed for real-time monitoring of event logs. Although during recent years several structured event log formats have been proposed, a significant part of event log messages remain unstructured or semi-structured. As an example, the widely used BSD *syslog* protocol [4] defines the priority, timestamp, hostname, and *syslog* tag (program name with its process ID) fields, whereas the message text,

which is the most informative part of the *syslog* message, remains a free-form string. Figure 1 depicts some example *syslog* messages without the priority, timestamp, and host-name fields (note that all *syslog* messages have been wrapped in Fig. 1 due to their length, and they appear in one line in real-life event logs).

As Fig. 1 illustrates, some parts of textual event log messages have a constant nature, while other parts are variables (such as IP addresses, process IDs, and port numbers). Template detection (or template mining) algorithms assume that each message in the event log is represented by a single line, and template detection involves reporting line patterns (templates) to the end user, which summarizes the content of the event log file. Each reported template should represent a group of event log messages, where constant parts shared by all messages are reported as is, and variable parts of messages are replaced with wildcards (in many past research papers, $\langle * \rangle$ has been used for denoting the wildcard, and we have adopted this notation in the current paper). In addition to example *syslog* messages, Fig. 1 also depicts relevant templates for similar messages. The mining of such templates is highly useful for several purposes, such as identifying event types in event logs, writing parsers (e.g., regular expression-

✉ Risto Vaarandi
risto.vaarandi@taltech.ee

Hayretin Bahşi
hayretin.bahsi@taltech.ee

¹ Centre for Digital Forensics and Cyber Security, Tallinn University of Technology, Tallinn, Estonia

² School of Informatics, Computing and Cyber Systems, Northern Arizona University, Flagstaff, USA

based parsing rules) for these event types, and preparing unstructured event log data for downstream machine learning tasks.

During the last 2 decades, a number of template detection algorithms have been proposed [5–23], and several comparative studies have been published for these algorithms [24–26]. Despite a significant amount of research in the domain of template mining, some research gaps have remained. Firstly, template detection approaches that have not employed LLMs (we call them *traditional algorithms* in the remainder of this paper) are unsupervised data mining-based approaches which can be tuned through hyperparameters. However, as pointed out in [21], selecting optimal values for hyperparameters is often not an easy task. Secondly, LLM-based template detection approaches are generally supervised and require labeled training data [19–22]. As discussed in [23], the creation of such training data is a laborious process, and training data sets need to be updated on the appearance of new event log message types.

Thirdly, many existing template mining approaches have various design limitations. As pointed out in recent studies [21, 26], several algorithms rely on preprocessing event log messages, which requires previous domain knowledge of message formats. Also, as discussed in [20, 26], a number of algorithms incorrectly assume that the log messages described by the same template always contain the same number of words. When that is not the case (for example, two Suricata IDS alert messages from Fig. 1 consist of 17 and 23 words, respectively), the algorithms fail to detect a common template for such messages.

Finally, existing studies have often used data sets which are either not recent or not related to cyber security. For example, data sets from the widely used Loghub repository [27] are not cyber security-centric, with several of them more than 15 years old.

The current paper addresses the aforementioned research gaps and proposes an unsupervised LLM-based template mining approach called LLM-TD for security event logs. In the paper, we have focused on the analysis of security related Linux *syslog* event logs, and the main contributions of the paper can be summarized as follows:

- Whereas several existing works have utilized public LLMs [20, 22, 23], security event logs usually contain sensitive data that can not be shared with external service providers. For this reason, the current paper studies the performance of LLM-TD with several small local LLMs.
- The paper uses five Linux security *syslog* data sets for evaluating the performance of LLM-TD, and releases these data sets publicly alongside with the implementation of LLM-TD (<https://github.com/ristov/llm-td>).

```
# sshd successful authentication messages and a template
sshd[12992]: Accepted password for william from 192.168.3.1
port 60676 ssh2
sshd[17837]: Accepted publickey for felix from 192.168.17.8
port 54097 ssh2

sshd[<*>]: Accepted <*> for <*> from <*> port <*> ssh2

# sshd failed authentication messages and a template
sshd[21378]: Failed none for alice from 192.168.3.250
port 60172 ssh2
sshd[6771]: Failed password for felix from 192.168.2.39
port 59020 ssh2

sshd[<*>]: Failed <*> for <*> from <*> port <*> ssh2

# Suricata IDS alert messages and a template
suricata[1815]: [1:2101411:13] GPL SNMP public access udp
[Classification: Attempted Information Leak] [Priority: 2]
{UDP} 192.168.130.19:55919 -> 192.168.153.226:161
suricata[19225]: [1:2034455:2] ET EXPLOIT Possible Gitlab
CE/EE Image Parser RCE Inbound (CVE-2021-22205)
[Classification: Attempted Administrator Privilege Gain]
[Priority: 1] {TCP} 192.168.1.7:3801 -> 192.168.81.29:80

suricata[<*>]: [<*>] <*> [Classification: <*>]
[Priority: <*>] {<*>} <*> -> <*>
```

Fig. 1 Example *syslog* messages and their templates

- The paper analyzes performance metrics that have been previously suggested for template detection algorithms [18, 24, 25], and proposes several heuristic principles for addressing the shortcomings of existing metrics.
- Finally, the paper highlights the advantages of LLM-based template detection over traditional algorithms, and discusses the importance of qualitative assessment of all identified templates. In particular, LLMs have the ability to infer correct templates from insufficient event log data and identify character patterns inside log message words. Furthermore, they are able to discover previously unknown knowledge from event logs, making the qualitative analysis of both correct and incorrect templates a relevant task.

The remainder of this paper is organized as follows—Sect. 2 discusses related work, Sect. 3 describes the LLM-TD method, Sect. 4 presents its evaluation, Sect. 5 discusses the evaluation results, and Sect. 6 concludes the paper.

2 Related work

Over the last 2 decades, several traditional template detection algorithms have been proposed. Traditional algorithms are generally unsupervised and assume that each event log message consists of tokens or *words* separated by delimiters (usually whitespace characters). The algorithms identify the words of variable nature (henceforth called *variable words*),

which allows the derivation of templates consisting of wildcards $\langle * \rangle$ and *constant words*.

SLCT [13] begins the template mining process by detecting frequent words from event log messages, continuing with creating cluster candidates based on the combinations of frequent words. Each candidate is represented by a template, and candidates that occur for at least s messages (s is a user-given threshold) are selected as clusters, with their templates being reported to the end user.

LogCluster [12] is a further development of SLCT that employs an advanced cluster candidate generation procedure. That procedure allows us to address the shortcomings of many other algorithms discussed in Sect. 1—templates detected by LogCluster can represent similar messages that contain different numbers of words. LFA [14] begins with identifying frequent words like SLCT and LogCluster and builds templates around frequent words that have similar occurrence times in event log messages.

LenMa [11] relies on the following observation—the constant words of the template have the same length (number of characters) in all messages matching that template, whereas words matching the wildcards do not have that property. Following that observation, LenMa converts event log messages into word length vectors, assigning messages with the same number of words into the same cluster based on the cosine similarity of their word length vectors. Each cluster has a template representative which is reported to the end user.

LKE [10] employs expert rules for removing variable parts from event log messages, and messages are then clustered based on their edit distance, deriving a template for each cluster. AEL [9] uses heuristic rules for identifying variable words in event log messages, dividing messages into bins based on the number of constant and variable words, and creating one or more templates for each bin. LogMine [15] first clusters event log messages by measuring their similarity in terms of common words and then generates a template for each cluster. These steps can be applied iteratively to produce more general templates from more specific ones.

IPLoM [7] is a divisive clustering algorithm which first splits the event log into clusters by the number of words in log messages. For each cluster, a word position is then identified with the smallest number of unique words, splitting the cluster by these words. Resulting clusters are split further by associations between word pairs. After that step, a template is derived for each cluster.

LogSig [6] converts each log message into word pairs and finds an optimal partition of messages into clusters, trying to maximize the number of common word pairs for messages in the same cluster. Spell [8] divides the event log messages based on the longest common subsequences (LCSs) of words, creating a new LCS object for a message if it is dissimilar to existing objects or merging it with the most similar object. Each LCS object contains information about the locations of

variable words in log messages corresponding to that object. After all messages have been processed, templates that are based on LCS objects are reported to the end user.

Logram [18] uses n -gram analysis for template detection, where each n -gram is a sequence of n words extracted from a log message. Logram begins with creating a dictionary of 2-grams and 3-grams, which holds their occurrence times. For template detection, Logram relies on the following observation— n -grams of constant words are frequent, while n -grams containing variable words are likely infrequent. To detect variable words, Logram first identifies infrequent 3-grams in event log messages and then analyzes infrequent 2-grams generated from infrequent 3-grams of the log message.

Drain [5] employs a tree structure called parse tree with a user-specified depth where templates are created and updated in leaf nodes. For all log messages with the same number of words, a separate child node is created under the root node of the parse tree. The nodes at the following levels contain message words and wildcards, allowing to search for a leaf node with a list of relevant templates for each log message. The similarity score between the message and the template is calculated to select the most suitable template for a log message, which has to exceed a user-given similarity threshold, and the selected template is updated (if no suitable template is found, a new template created).

SHISO [17] uses a tree structure for clustering log messages on the fly, and n -gram analysis for adjusting templates during log message processing. MoLFI [16] employs a genetic algorithm for generating templates from event log messages.

Unlike previously described traditional algorithms, LLM-based template mining approaches are generally supervised. LogPPT [21] harnesses the Adaptive Random Sampling algorithm for selecting training data points from preprocessed event log data. Training data is then labeled with ground truth templates by the human analyst and used to *fine-tune* (train) a pretrained RoBERTa language model that will be applied for template detection. The authors evaluated the model performance on 2000-message labeled event logs from the Loghub repository.

LLMParser [19] employs four pretrained open-source LLMs fine-tuned with labeled examples selected from preprocessed event log data. For example selection, Mean Shift algorithm is used to cluster the preprocessed event log, and examples are sampled from detected clusters. The performance of models was measured on event logs of 2000 messages from the Loghub repository. The authors also evaluated the performance of *in-context learning*, where instead of fine-tuning, a *prompt* (instructions in natural language) is presented to an LLM on how to address the template mining task (please see Fig. 2 in Sect. 3 for an example prompt). According to the results in [19], fine-tuning yielded higher

performance than in-context learning. As discussed in [19], used LLMs accepted prompts of limited size, and insufficient log parsing instructions with a very few examples had a negative impact on performance. In contrast, fine-tuning enabled the model to learn from a larger number of more diverse examples, yielding a better performance.

DivLog [20] uses in-context learning for template detection, harnessing the public GPT-3 LLM through the OpenAI interface. The algorithm samples a subset of event log data with the Determinantal Point Process algorithm, and selected log messages are labeled by the human analyst. To detect a template for the log message, DivLog builds a prompt of similar message-template pairs selected from the labeled data so that LLM can learn from the provided examples to find the right template for the given message. The performance of DivLog was evaluated on 2000-message event logs from Loghub.

LILAC [22] uses in-context learning with the public ChatGPT LLM through the OpenAI interface, where the human analyst needs to label a subset of event log messages for prompt-building purposes. For selecting messages for labeling, a hierarchical clustering algorithm is applied for the event log, sampling messages from different clusters based on a quota distribution method. When LILAC needs to detect a template for the log message, it first checks its template cache for a suitable template in order to minimize queries to LLM. Such a design allows to apply LILAC for processing large event logs, and the authors evaluated the performance of LILAC on full-size event logs (i.e., not 2000-message event log chunks as in several other works [19–21]) from Loghub.

Similarly to LILAC, Lunar [23] employs ChatGPT LLM through OpenAI interface. Unlike previously discussed LLM-based algorithms which are supervised [19–22], Lunar is an unsupervised method and does not rely on example log messages labeled by a human analyst with ground truth templates. Whereas other approaches select specific labeled log message examples for building prompts, so that a template could be detected for *one log message* according to labeled examples, Lunar employs a different heuristic—if *several similar log messages* are submitted to LLM, LLM is able to find a common template for them without labeled examples. Ideally, log message groups submitted to LLM should match the same template, while being diverse enough so that LLM can distinguish variable parts from constant parts in log messages. For example, in the first message group in Fig. 1, all variable parts of two log messages are different, which eases the template detection task for LLM.

Lunar employs a hierarchical log clustering algorithm to detect groups of similar log messages. First, like many traditional algorithms, Lunar assumes that log messages matching the same template contain the same number of words, and the event log is thus split into buckets by the number of words in log messages. By the design of Lunar, the further

processing of each bucket is independent of other buckets, which allows the execution of these tasks in parallel. In each bucket, top- k frequent words are identified, creating clusters for log messages that share the same frequent words, and merging clusters with too few log messages based on top- $(k - 1)$ frequent words. To select message groups for LLM, Lunar samples so-called log contrastive units (LCUs) from each cluster, so that log messages in LCUs would be diverse enough. If a template is successfully detected for an LCU, all messages matching this template are removed from buckets. The sampling of LCUs from buckets and querying a common template for them from LLM continues until all buckets are empty.

3 Unsupervised LLM-based template detection

In this section, we will describe an unsupervised LLM-based template detection method (henceforth called LLM-TD) that has been motivated by research gaps in existing literature. As discussed in Sect. 2, unsupervised LLM-based template detection has received little attention, apart from a recent work proposing Lunar [23]. However, Lunar suffers from a design limitation discussed in Sect. 1, assuming that log messages matching the same template have the same number of words (see Sect. 2 for more details). Because unsupervised template detection does not need a larger pool of log message examples labeled with ground truth templates, it allows to eliminate the labeling workload of human analysts, and LLM-TD is therefore employing this template detection paradigm.

Secondly, several existing works [20, 22, 23] employ public LLMs through the OpenAI interface, which implies submitting event log data to an external service provider. However, security logs are likely to contain highly sensitive information (e.g., information about user accounts and authentication methods in an organization), and sharing it with external parties is regarded as a violation of security policies.

Whereas local LLMs allow processing event logs without external service providers, such LLMs are known to be less performant than public LLMs. For this reason, their use has been primarily considered with fine-tuning [19, 21], which, according to a recent study, yields better performance than in-context learning [19]. However, we argue that fine-tuning requires a considerable amount of machine learning expertise, which is not always present in an organization.

For the reasons above, this paper explores the use of local LLMs for security event log analysis, and the LLM-TD method employs an unsupervised in-context learning paradigm for template detection. Whereas other methods attempt to detect *one* template with each LLM query (i.e.,

templates are detected one-by-one), LLM-TD uses a different approach, which is able to identify *several* templates with one query. In order to achieve that, LLM-TD utilizes a prompt that formulates the task of a template detection for a small event log which contains messages matching *several templates*, not for one message or a group of similar messages which match *one template only* (Fig. 2 depicts the prompt used by LLM-TD).

When compared to Lunar, which is another unsupervised LLM-based algorithm (see Sect. 2), both LLM-TD and Lunar process a batch of several log messages with one LLM query because it can ease the unsupervised template detection task for LLMs [23]. However, Lunar harnesses a dedicated multi-stage selection method for ensuring the similarity of messages within a batch, so that a single template can be detected for the batch. In contrast, as LLM-TD does not aim to detect one template only which must cover the entire batch, it can process less similar messages with one LLM query, and thus does not need a complex method for creating message batches. Note that LLM-TD does not specify any constraints to the number of detected templates in LLM query, and LLM is free to report back *any* number of templates (e.g., one, two, five, etc.) as it sees most appropriate for the submitted batch.

As pointed out in [23], unsupervised detection of templates is complicated if too diverse log messages are submitted to LLM. Following that observation, LLM-TD divides the event log data into different parts by applications (e.g., sshd, Suricata) that have produced the log messages, where each part is processed independently. Similarly to Lunar, dividing the event log data allows for parallel processing but does not restrict templates to match the messages with the same number of words (unlike Lunar). Having each part containing messages from the same application leads to much less diversity among log messages and fewer templates that need detection.

Also, in the case of Linux *syslog*-based security logs (the primary focus of this paper), the application name can be retrieved from the *syslog* tag field which is defined by the BSD *syslog* protocol [4]. Since the purpose of LLM-TD is the analysis of unstructured log message data, LLM-TD implementation will not consider timestamp and hostname fields of *syslog* messages, targeting the *syslog* tag field which is followed by a free-form textual message string (Fig. 1 depicts such *syslog* message examples). Note that the location of timestamp, hostname, *syslog* tag (application name), and message string fields are well documented in the *syslog* protocol standard [4], and therefore the identification of these fields does not assume detailed domain knowledge about the individual message formats of logging applications.

After event log data has been divided into parts by the name of the logging application, Algorithm 1 is applied to each part (that is, an event log of an individual application). LLM-TD algorithm makes two passes over the

Algorithm 1 Event Log Analysis with LLM-TD

Input: $L = (l_1, \dots, l_n)$ – event log of n messages,
 k – batch size
Output: T – templates, U – uncovered messages,
 V – duplicate templates

```

1:  $T \leftarrow \emptyset$ 
2:  $B \leftarrow ()$ 
3: for each  $l \in L$ 
4:   if  $\exists t \in T$ , so that  $l$  matches  $t$  then next loop iteration
5:   append  $l$  to  $B$ 
6:   if  $|B| < k$  then next loop iteration
7:    $X \leftarrow \text{QueryLLMforTemplates}(B)$ 
8:    $\text{Merge}(T, X)$ 
9:    $B \leftarrow ()$ 
10:  if  $|B| > 0$  then
11:     $X \leftarrow \text{QueryLLMforTemplates}(B)$ 
12:     $\text{Merge}(T, X)$ 
13:   $U \leftarrow \emptyset$ 
14:   $Y \leftarrow \emptyset$ 
15:  for each  $l \in L$ 
16:     $M \leftarrow \{t \mid t \in T, l \text{ matches } t\}$ 
17:    if  $|M| = 0$  then  $U \leftarrow U \cup \{l\}$ 
18:    if  $|M| = 1$  then  $Y \leftarrow Y \cup M$ 
19:   $V \leftarrow T \setminus Y$ 
20: return  $T, U, V$ 

```

event log, where the first pass (lines 1–12 in Algorithm 1) involves template detection, and the second pass (lines 13–19 in Algorithm 1) the identification of duplicate templates and log messages not covered by templates, allowing the human expert to analyze challenging messages and potentially redundant templates manually.

During the first pass, log messages are processed sequentially, submitting them to LLM in batches of k messages. If a log message matches an already detected template in the template set T , it is skipped (line 4 in Algorithm 1) for decreasing the number of computationally expensive queries to LLM (for matching the message with the template, the regular expression representation of the template is used). After the batch of k messages has been assembled for processing by LLM, the batch is included in a prompt that is illustrated in Fig. 2. After the prompt has been created, it is used for querying the LLM (line 7 in Algorithm 1).

The prompt consists of a static and dynamic part, with the first containing a hardcoded example of how to detect multiple templates from a small log file, whereas the dynamic part contains the message batch that needs processing. As a response to this prompt, LLM provides an answer in natural language (see Fig. 2). In order to pick up template definitions from LLM output, it is processed with regular expression-based filters, and for each definition, a corresponding regular expression is created automatically (see the template candidate box in Fig. 2). To check if the template definition is valid for the given message batch, its regular expression is matched against the messages, and the template is discarded if there is no match. Otherwise, the template is regarded as

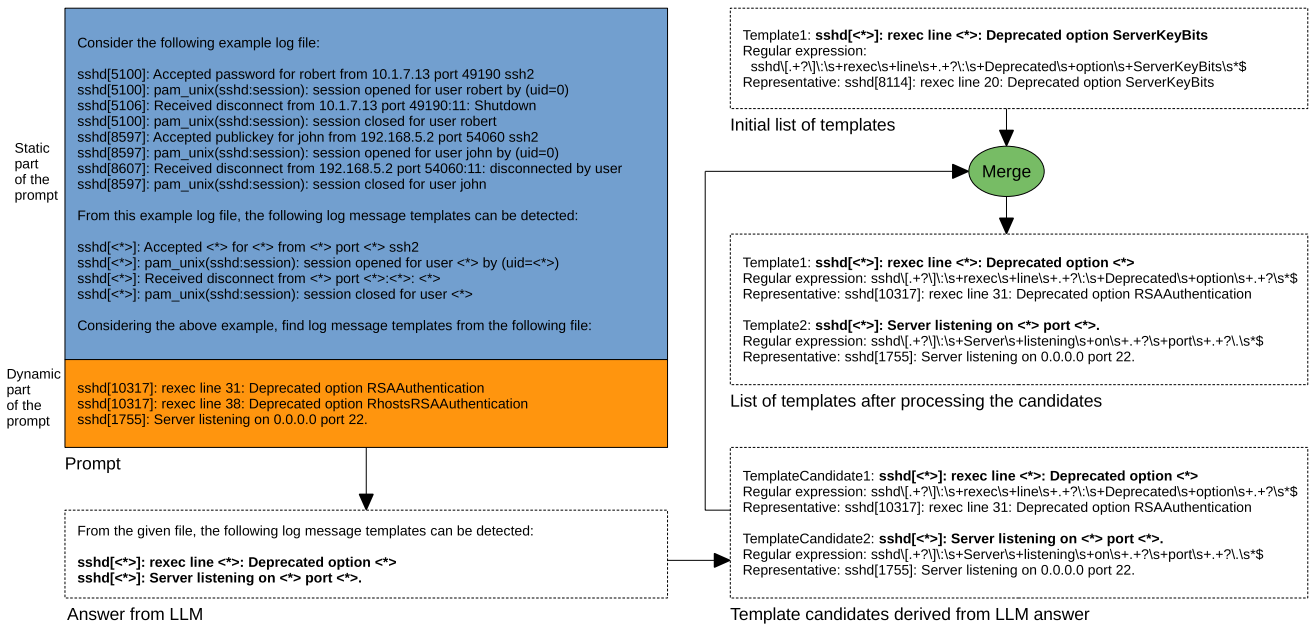


Fig. 2 Interaction between LLM-TD and the underlying LLM (*QueryLLMforTemplates* and *Merge* procedures from Algorithm 1)

a *template candidate* and the first matching message in the batch is considered its *representative*.

After the candidates have been identified, they are merged with the set of templates T (line 8 in Algorithm 1). The process of merging has been illustrated in Fig. 2. Firstly, candidate definitions already present in T are dropped, adding the remaining definitions to T . After that, all definitions in T are compared by matching their representatives in order to detect and drop definitions that are more specific than others. That step is taken to increase the computational efficiency since templates in T are employed as filters for event log data (line 4 in Algorithm 1). For example, in Fig. 2, the representative of the template *sshd[<*>]: rexec line <*>: Deprecated option ServerKeyBits* matches the candidate template *sshd[<*>]: rexec line <*>: Deprecated option <*>*. On the other hand, the representative of the candidate template does not match the template *sshd[<*>]: rexec line <*>: Deprecated option ServerKeyBits*. Therefore, the template *sshd[<*>]: rexec line <*>: Deprecated option ServerKeyBits* will be dropped from T .

After the first pass over the event log is complete, LLM-TD checks for the presence of an incomplete batch with less than k messages and, if found, submits it to LLM (lines 10–12 in Algorithm 1). During the second data pass (lines 13–19 in Algorithm 1), event log messages not matching any templates from T are identified, assigning them to set U . In addition, duplicate templates are highlighted by assigning them to set V . A template is regarded as a duplicate if no event log message is matched by this template alone. For instance, if all successful SSH public key-based logins originate from the

host 10.1.1.1, all such messages always match both of the following templates, which makes these templates duplicates:

```

sshd[<*>]: Accepted publickey for <*> from <*> port <*>
ssh2
sshd[<*>]: Accepted <*> for <*> from 10.1.1.1 port <*>
ssh2

```

Therefore, highlighting them will help the human expert to derive a correct template from them manually:

```

sshd[<*>]: Accepted <*> for <*> from <*> port <*> ssh2

```

4 Evaluation

4.1 Experiment environment and data sets

For conducting the experiments with LLM-TD on Linux security *syslog* files, we created an implementation of LLM-TD in Perl, with the *QueryLLMforTemplates* procedure from Algorithm 1 being implemented in Python using the LangChain framework. For local LLMs, we selected three local LLMs—OpenChat [28], Mistral [29], and Wizardlm2 [30]. In the case of some benchmarks [28], OpenChat has been reported to offer comparable performance to ChatGPT, which has been employed by several previous works for template detection [22, 23]. Also, Mistral and Wizardlm2 are state-of-the-art LLMs from Mistral AI and Microsoft AI, respectively, which we found to work well for template detection tasks. We also considered a number of other local LLMs for general purposes (Llama2, Llama3, Mixtral, Phi, and Phi3) and more specific goals such as code analysis and mathematical reasoning (Codellama, Codeup, and Wizard-

math), but they featured lower runtime and template detection performance than the selected three LLMs.

For all three models, we employed their 7B implementations through the Ollama framework, which was installed on a Rocky Linux 9 workstation with NVIDIA GeForce RTX 2080 Ti video card, Intel Core i9-10900X CPU, and 128 GB of memory. Therefore, the evaluation allowed us to establish how well small LLMs with parameter size 7B would be suited for mining templates from security event logs using the computing power of only one workstation.

In order to find how well LLM-TD with small LLMs is performing in comparison to traditional algorithms, we selected Drain [5] as a baseline because it was identified as the most performant traditional algorithm in the study by Zhu et al. [24]. Also, Drain has been used as a baseline in several recent studies on LLM-based template detection [19–23]. We were not able to include Lunar (the only existing unsupervised LLM-based algorithm) in the comparison since its implementation is not publicly available.

Five medium-sized Linux *syslog* data sets labeled with ground truth templates were used for evaluation, with the data sets being described in Table 1. The medium size of data sets eased the task of qualitative analysis of detected templates (the importance of this task is clarified in Sect. 4.3.3).

The data sets covered different areas of network and system security: network intrusion detection (*suricata* data set), authentication and access over SSH and SNMP protocols (*sshd* and *snmpd* data sets, respectively), local authentication (*su* data set), and web application code quality issues (*apache* data set). Each data set contained messages from one application only—*sshd* daemon messages from 37 hosts, *su* authentication messages from 28 hosts, *Suricata* network IDS messages from one host, *snmpd* daemon messages that reflected SNMP-based access to 6 hosts, and *Apache* web server messages about code quality issues of PHP applications running on one host.

All data sets were collected in a large academic institution from a production environment (i.e., data sets were not created in a lab environment through some experiments, but reflect real-life log data of an organizational IT system). The *sshd*, *su*, *snmpd*, and *apache* data sets were collected during 35 days, and the *suricata* data set during 5 days. The *suricata* data set originated from a *Suricata* IDS which was monitoring an external network perimeter of the entire organization and routinely processing 2–3 Gbit/s of network traffic. The *apache* data set was collected from publicly accessible organizational web portal, whereas remaining data sets originated from system daemons of critical Linux hosts.

During the experiments, we ran Drain with similarity threshold and parse tree depth parameters set to 0.39 and 6, respectively (the same settings were used for *syslog* data in the comparative study by Zhu et al. [24]). Due to the unsupervised nature of the LLM-TD algorithm, it was used with

```
# an example event log with two messages

sshd[12992]: Accepted password for william from 192.168.3.1
port 60676 ssh2
sshd[17837]: Accepted password for felix from 192.168.17.8
port 54097 ssh2

# template detected by LLM-based algorithm A
sshd[<*>]: Accepted password for <*> from <*> port <*> ssh2

# template detected by traditional algorithm B
<*> Accepted password for <*> from <*> port <*> ssh2

# template detected by traditional algorithm C
<*> Accepted password <*> <*> <*> <*> port <*> ssh2

# ground truth
sshd[<*>]: Accepted <*> for <*> from <*> port <*> ssh2
```

Fig. 3 An example event log with detected templates and ground truth

the *same* static part of the LLM prompt (the blue part of the prompt in Fig. 2 from Sect. 3) for *all* data sets from Table 1.¹

4.2 Performance metrics

In the research literature, several different metrics have been proposed for assessing the performance of template detection algorithms, and a recent study by Khan et al. [25] provides a detailed treatment of these metrics. To support the discussion in this section, an example event log in Fig. 3 is provided, which assumes that the same template describes all messages in the log.

The *grouping accuracy* [24] is a metric that measures the proportion of correctly parsed messages in the event log. A message is regarded as correctly parsed if its template corresponds to the same group of messages as defined by the ground truth. For example, in Fig. 3 algorithms A, B, and C have created one template for two messages found in the event log, and have thus reached the same grouping as ground truth. However, as pointed out in recent studies [18, 25, 26], grouping accuracy does not reflect if detected templates are accurately reflecting constant and variable parts of log messages (e.g., the result from algorithm C in Fig. 3 reports several constant parts as variables).

The *parsing accuracy* [18] measures the proportion of correctly parsed messages in the event log and regards the message as correctly parsed if its template identifies all constant and variable parts correctly. In [18], Dai et al. conducted the manual analysis of messages and templates for establishing the parsing accuracy. Considering the ground truth, no algorithm in Fig. 3 has parsed the messages correctly.

Grouping and parsing accuracy have one weakness—if only a small part of templates cover the majority of the

¹ In contrast, with supervised methods the template detection examples (see the blue part of the prompt in Fig. 2) would no longer be static, but dynamically created for each log message batch, selecting the most appropriate examples from a larger pool previously developed by human experts. With unsupervised LLM-TD, the creation of such a large pool of labeled examples is not needed.

Table 1 Event log data sets

Data set	Number of log messages	Number of ground truth templates	Number of logging hosts
sshd	632,677	36	37
su	2354	13	28
suricata	999,622	6	1
snmpd	108,935	7	6
apache	63,947	7	1

event log, detecting these templates correctly will yield high accuracy, although they represent a small minority among templates. To address this issue, Khan et al. [25] have proposed to consider detected templates with equal weight and have suggested the concept of *template accuracy*. According to this concept, a detected template t is correctly identified if all log messages that the template t matches correspond to a single (i.e., not more than one) ground truth template v , so that templates t and v are identical. As seen from Fig. 3, none of the three detected templates is correctly identified since the messages they match correspond to a different ground truth template.

However, existing metrics face several challenges illustrated by Fig. 3. Firstly, although in Fig. 3 the word *password* is actually a variable part in log messages and denotes the authentication method (see also Fig. 1 in Sect. 1), there are no other authentication method keywords present in log data. This complicates the identification of the ground truth template, and traditional algorithms are generally unable to handle this task. Therefore, a performance metric should take into account the nature of event log data, and if some $\langle * \rangle$ in the ground truth template is always matching the same string s in event log data, a template where s appears in place of that $\langle * \rangle$ should be regarded as correctly identified. Following that logic, the template detected by algorithm A is correct since the second $\langle * \rangle$ in the ground truth template always matches the same string (*password*) as reported in the detected template. In the remainder of the paper, we use this principle (henceforth called P1) to decide if a detected template is correct.

A simplistic workaround to the aforementioned problem would be to adjust the ground truth template according to the event log data (e.g., in Fig. 3 that would involve using the template from algorithm A as ground truth). Whereas that approach would indeed be appropriate for traditional algorithms, LLM-based solutions are, in some cases, able to correctly infer $\langle * \rangle$ even from constant data. In the rest of the paper, we call this task *inferring the ground truth template from insufficient event log data*. Therefore, adjusting the ground truth templates would not allow to adequately assess the capabilities of LLM-based algorithms for addressing this task. To summarize, the P1 principle allows the

evaluation of traditional and LLM-based algorithms, taking into account the limitations of traditional algorithms. However, such evaluation should also qualitatively assess which correct templates are more similar to the ground truth templates for the given event log data.

The second challenge to the existing metrics comes from another limitation of traditional algorithms. Namely, most algorithms assume that each event log message consists of words that are separated by delimiters (usually whitespace characters), and no attempt is made to analyze the content of words by individual characters. As a result, the words *sshd[12992]:* and *sshd[17837]:* from Fig. 3 are identified as variables by many algorithms and reported as $\langle * \rangle$, with a template from algorithm B illustrating that problem.

For a workaround, two approaches have been suggested, with the first being the preprocessing of event log data (e.g., by replacing variable substrings in words with $\langle * \rangle$) [24]. However, as discussed in Sect. 1, preprocessing requires detailed knowledge of message formats. Also, Khan et al. have proposed the modification of ground truth templates with heuristic rules [25], which can replace a template word with $\langle * \rangle$ if that word contains $\langle * \rangle$ as a substring. For example, *sshd[$\langle * \rangle$]:* would be converted to $\langle * \rangle$ in the ground truth template from Fig. 3, making it identical to the template detected by algorithm B. Similarly to the workaround for the first challenge to performance metrics, it would be appropriate for traditional algorithms. However, LLM-based algorithms are able to identify variable parts inside words, and the adjustment of ground truth would downplay the strengths of LLM-based methods.

As a solution for this problem, we have used the following principle (henceforth called P2), which takes into account the limitations of traditional algorithms. If some word in the ground truth template contains $\langle * \rangle$ as a substring, a detected template where $\langle * \rangle$ appears in place of that word is regarded correct, provided that this template does not match more messages than the ground truth template (i.e., replacing the entire word with $\langle * \rangle$ does not have any impact on the given event log data). Note that we have applied principle P2 *only to traditional algorithms* (i.e., Drain) in this paper. According to this principle, the template detected by algorithm B in Fig. 3 is correct.

In addition to using principles P1 and P2 for assessing the correctness of an individual template, this paper follows the example of previous studies [22, 23] and employs the F1-score metric (harmonic mean of precision and recall) for measuring the performance of algorithms. *Precision* is defined as the ratio of correctly identified templates and all detected templates, whereas *recall* is the ratio of correctly identified templates and all ground truth templates. Also, F1-score is defined as:

$$F1 = 2 * \frac{precision * recall}{precision + recall} \quad (1)$$

4.3 Results

This section presents the evaluation results for four selected approaches (LLM-TD with three LLMs and Drain). Section 4.3.1 discusses the execution time of evaluated approaches and the impact of log message batch size to the runtime of LLM-TD. Section 4.3.2 covers the template detection performance and Sect. 4.3.3 provides the analysis of incorrectly detected templates.

4.3.1 Execution time

For measuring the execution time and performance of four different template detection algorithms described in Sect. 4.1 (Drain and LLM-TD with OpenChat, Mistral, and Wizardlm2), we applied them to data sets from Table 1 10 times. Repeated execution allowed us to estimate the execution time of algorithms better, and since an LLM can produce different output for the same input data, we were able to estimate how the number of detected templates varies for LLM-based approaches. Since the implementations of Drain and LLM-TD are single-threaded, the number of CPUs of the experiment machine did not influence the execution time. Table 2 shows the results for 10 experiment iterations. Note that in Table 2 and the remainder of the paper, we have used the name of an LLM to denote LLM-TD using that LLM (e.g., OpenChat denotes LLM-TD with OpenChat). Also, if an algorithm was not able to process a data set within 10h, respective experiment was terminated and corresponding cell in Table 2 contains the ‘-’ character.²

Our first notable finding was that although LLMs are known to be non-deterministic, the LLM-TD algorithm produced the same result for all 10 iterations of every experiment scenario. In other words, LLM-TD featured a deterministic

behavior during all experiments, making it similar to traditional algorithms which are usually deterministic.

We also assessed the influence of the log message batch size (parameter k in Algorithm 1) to the speed of the LLM-TD algorithm for the following settings: $k = 2, 5, 10, 20$ (i.e., batch size was increased by 2–2.5 times for the next scenario). As Table 2 shows, OpenChat featured short execution times of 5–15 s for three smaller data sets (*su*, *snmpd* and *apache*) for all values of k . For two larger and more challenging data sets (*sshd* and *suricata*), increasing the value of k from 2 to 10 decreased the runtime, whereas increasing k from 10 to 20 provided no further performance benefits. For example, with $k = 10$ the *suricata* data set was processed in 1.5 min, while with $k = 20$ the runtime exceeded 10h.

For Mistral, only the smallest *su* data set featured short execution times (13–44 s) for all values of k . For the *snmpd* and *apache* data sets, $k = 2$ provided the worst runtime performance. Interestingly, for the largest *suricata* data set the opposite pattern was observed, and $k = 2$ was the only setting which led to a runtime of less than 10h (about 42 min). However, when investigating this phenomenon, we discovered it was caused by the detection of an overly general template in early phases of event log processing, which caused the exclusion of most log data from further processing (see line 4 in Algorithm 1). We will provide a detailed discussion of that issue in Sect. 4.3.3. Apart from the *suricata* and *su* data set, increasing the value of k until 10 provided significant performance benefits for Mistral, and with $k = 10$ all data sets except *suricata* were processed in less than 1 min. However, increasing k from 10 to 20 led to a poor runtime performance of over 2h on the *sshd* data set.

Like Mistral, Wizardlm2 was able to process the *suricata* data set in 10h only with $k = 2$, and the reasons for this were the same as for Mistral (the detection of an overly general template). Apart from the *sshd* data set, increasing k from 2 to 5 provided less noticeable runtime performance benefits than observed for Mistral. However, increasing k further to 10 decreased the runtime significantly for *sshd*, *snmpd*, and *apache* data sets, with all data sets except *suricata* being processed within 1.5–30 min. Increasing the value of k from 10 to 20 improved performance on *sshd* and *su* data sets, while leading to a longer runtime on the *snmpd* data set.

According to the analysis above, $k = 10$ yielded the best overall performance for LLM-TD with OpenChat and Mistral, and for Wizardlm2 both $k = 10$ and $k = 20$ featured a good performance. Because $k = 10$ also yielded the best F1-scores for the majority of experiment scenarios, further experiments described in this paper were conducted with this particular setting (i.e., the log message batch size of 10).

As Table 2 illustrates, Drain was the fastest algorithm for all data sets, having an average execution time of less than 20s in all cases. With $k = 10$, OpenChat was the second fastest on most data sets, running for less than 91 s on all data

² This allowed to limit the overall duration of experiments to 12 weeks, because 10 iterations would take at most 100h, and evaluating four algorithms on five data sets would thus require at most 2000h, which is about $2000/24 \approx 83.33$ days ≈ 12 weeks.

Table 2 Execution time of template detection methods

Method	Average runtime in seconds (standard deviation in parentheses)					Number of detected templates (the number of uncovered messages in parentheses)				
	sshd	su	suricata	snmpd	apache	sshd	su	suricata	snmpd	apache
OpenChat	175.136	15.494	1195.762	7.086	6.984	29	1	22	7	7
$k = 2$	(0.334)	(0.017)	(2.64)	(0.031)	(0.021)	(189)	(0)	(249)	(0)	(0)
OpenChat	77.413	8.511	443.732	6.274	6.742	28	1	43	7	7
$k = 5$	(0.265)	(0.011)	(1.214)	(0.032)	(0.018)	(148)	(0)	(58)	(0)	(0)
OpenChat	48.644	12.645	90.544	5.868	12.609	31	13	15	7	7
$k = 10$	(0.44)	(0.019)	(0.737)	(0.038)	(0.091)	(4)	(1)	(2)	(0)	(0)
OpenChat	85.694	13.953	–	6.395	6.364	27	13	–	7	7
$k = 20$	(0.395)	(0.011)		(0.785)	(0.017)	(223)	(0)		(0)	(0)
Mistral	309.399	21.925	2542.645	7154.681	3745.85	32	2	2	5	2
$k = 2$	(0.505)	(0.008)	(0.71)	(4.854)	(0.835)	(43)	(0)	(102)	(0)	(0)
Mistral	166.244	44.358	–	25.052	5.262	23	10	–	7	1
$k = 5$	(1.083)	(0.022)		(1.073)	(0.004)	(54)	(39)		(4)	(0)
Mistral	59.088	17.51	–	26.402	8.088	25	10	–	1	2
$k = 10$	(0.433)	(0.004)		(0.071)	(0.01)	(65)	(6)		(0)	(0)
Mistral	7544.625	13.787	–	10.737	7.32	22	10	–	5	2
$k = 20$	(2.382)	(0.015)		(0.785)	(0.008)	(204)	(6)		(12)	(0)
Wizardlm2	–	3176.84	8540.346	873.894	7771.931	–	75	1	8	4
$k = 2$		(1.084)	(3.212)	(0.158)	(5.215)		(1163)	(0)	(10)	(2)
Wizardlm2	5400.69	1351.209	–	1042.132	8119.098	56	22	–	6	8
$k = 5$	(1.528)	(0.155)		(0.766)	(1.593)	(491)	(244)		(835)	(8245)
Wizardlm2	1824.513	1309.906	–	93.4	324.047	51	9	–	6	7
$k = 10$	(0.591)	(0.162)		(1.243)	(0.086)	(1121)	(822)		(12)	(15)
Wizardlm2	906.329	654.405	–	186.198	323.856	20	20	–	6	7
$k = 20$	(0.737)	(0.153)		(1.584)	(0.062)	(957)	(927)		(12)	(15)
Drain	9.904	0.091	19.696	1.662	1.065	44	16	80	8	4
	(0.037)	(0.003)	(0.151)	(0.012)	(0.008)	(0)	(0)	(0)	(0)	(0)

Best results highlighted in bold

sets, and was the only LLM-based approach that was able to process all data sets successfully. Mistral and Wizardlm2 did not complete the analysis of the *suricata* data set in 10h for most settings of k . On the other four data sets, Wizardlm2 was usually the slowest approach – for example, with $k = 10$, it had an execution time of about 5, 22, and 30 min on *apache*, *su*, and *sshd* data sets, respectively.

When investigating the causes of execution time differences of LLM-TD with different LLMs, we discovered the number of LLM queries as one of the reasons, and Table 3 shows the relevant data for $k = 10$ (for the sake of brevity, data for other values of k has been omitted, because it followed the same pattern as data in Table 3). As Table 3 illustrates, OpenChat and Mistral were able to process all data sets with only 3–24 queries, which involved a modest amount of total LLM query time. However, Wizardlm2 executed noticeably more queries for *apache*, *su*, and *sshd* data sets, and that

resulted in a significantly more time spent for running these queries.

The need for a larger number of queries is connected to the fact that Wizardlm2 generated more invalid templates than other LLMs (e.g., 56–316 invalid templates for *apache*, *su*, and *sshd* data sets, whereas OpenChat and Mistral generated 1–34 invalid templates for these data sets). As discussed in Sect. 3, LLM-TD validates each template returned from LLM by checking if this template matches at least one message in the message batch which LLM had to process. Because Wizardlm2 made more mistakes in the template detection process, that process became much slower (e.g., LLM-TD was able to skip less messages in line 4 of Algorithm 1), leading to longer execution times.

Another reason for execution time differences was the response time for LLM queries. Based on the data from Table 3, the average response time of OpenChat was 1.7–2.3 s on the *sshd*, *su*, *snmpd*, and *apache* data sets, and 2.8 s on the *suri-*

cata data set. For Mistral, the average response time on the *sshd*, *su*, *snmpd*, and *apache* data sets was 1.9–2.5 s, whereas Wizardlm2 (the slowest LLM) had the average response time of 4.1–9.3 s on these data sets. Therefore, a noticeably longer LLM query response time of Wizardlm2 was another factor behind its longer execution times.

Due to long runtimes of Mistral and Wizardlm2 on the *suricata* data set, we were not able to execute them 10 times as for other scenarios shown in Table 3, and we conducted only one experiment iteration for assessing the impact of the number of LLM queries, the average query response time, and the number of invalid templates on the execution time. Mistral produced 23,210 invalid templates during 15,700 LLM queries with an average query response time of 5.1 s, which led to the runtime of $22\frac{1}{4}$ hours (i.e., almost 1 day). Wizardlm2 produced 8577 invalid templates during 42,099 LLM queries with an average query response time of 9.7 s, having the runtime of 115 h (i.e., almost 5 days). In both cases, vast majority of the runtime was spent for LLM queries. These results illustrate that a larger number of invalid templates are likely to involve a larger number of LLM queries, and when combined with longer query response times, the overall impact on the execution time is significant. It should be noted that long execution times on the *suricata* data set did not lead to a good template detection performance, and the F1-scores of Mistral and Wizardlm2 remained below 0.02.

4.3.2 Template detection performance

As discussed in the previous section, log message batch size 10 ($k = 10$) tended to yield the best execution times and F1-scores for LLM-based approaches, and this section presents template detection results for this setting. Table 4 shows the template detection performance in terms of F1-scores for all evaluated methods.

As for the number of detected templates for 10 experiment iterations, Drain always produced the same result since it is a deterministic algorithm. Also, by its design, Drain assigns all messages in the event log into groups, creating one-message groups for rare messages and thus leaving no messages unprocessed. As mentioned in the previous section, LLM-TD produced the same output during all experiment iterations, behaving similarly to Drain despite the non-deterministic nature of LLMs. However, the templates detected by LLM-TD might not cover the entire event log, and the number of uncovered messages was smallest for OpenChat (four messages for the *sshd* data set and less for others, see the row for OpenChat with $k = 10$ in Table 2). In contrast, Wizardlm2 produced the largest number of uncovered messages for all data sets (e.g., in the case of *sshd* and *su* data sets 1121 and 822, respectively, which complicates their manual analysis).

As discussed in Sect. 3, one notable difference of LLM-TD from other LLM-based template detection algorithms is its ability to identify multiple templates with one LLM query. This allows to reduce computationally expensive interaction with LLM, and we observed the effect of that property for OpenChat on three data sets. According to Tables 3 and 4, OpenChat was able to detect 25 correct templates with 19 queries on the *sshd* data set, 10 correct templates with 7 queries on the *su* data set, and 7 correct templates with 3 queries on the *snmpd* data set. Mistral featured a similar behavior on the *su* data set. In contrast, because other LLM-based algorithms can identify at most one template with a single query, the detection of m correct templates will require at least m LLM queries, involving an additional computational overhead.

According to Table 4, OpenChat achieved the best performance on four data sets, whereas Mistral had the highest F1-score on the *su* data set. The *suricata* data set was the most challenging for all evaluated approaches since Mistral and Wizardlm2 were not able to process it within 10 h. For $k = 2$ (the only setting where the processing of the *suricata* data set completed in 10 h), the F1-scores of Mistral and Wizardlm2 were 0.25 and 0, respectively. Furthermore, OpenChat and Drain achieved low F1-scores of 0.286 and 0.07 on the *suricata* data set. The vast majority of the log messages in this data set were Suricata IDS alerts with different numbers of words that correspond to one ground truth template (see Fig. 1 in Sect. 1), and this complicated the detection of ground truth templates for all evaluated approaches. Another challenging data set was *apache*, where only OpenChat achieved the F1-score of 0.714, whereas F1-scores of other approaches remained below 0.5.

From the evaluated four methods, performance of Mistral and Wizardlm2 was noticeably lower. For example, Wizardlm2 exceeded the F1-score of 0.5 only in the case of the *snmpd* data set. Also, Mistral yielded the F1-score of 0 on *snmpd* and *apache* data sets (the reason was the detection of overly general patterns, an issue which was already mentioned in the previous section and will be discussed in details in Sect. 4.3.3).

Although $k = 10$ yielded the best F1-scores for the majority of experiment scenarios, we investigated if other values of k would significantly improve the lower F1-scores from Table 4. The F1-score of Mistral increased from 0 to 0.714 on the *snmpd* data set with $k = 5$, whereas no improvement was observed for the *apache* data set. As for Wizardlm2, its F1-score increased from 0.091 to 0.4 on the *su* data set with $k = 5$, and from 0.207 to 0.357 on the *sshd* data set with $k = 20$. For other scenarios no performance improvements were observed.

As discussed in Sect. 4.2, we have used principle P1 to assess the correctness of detected templates. However, not employing this principle would allow us to assess the capa-

Table 3 Interaction with LLM (log message batch size 10)

Method	Number of LLM queries					Total LLM query time (standard deviation in parentheses)				
	sshd	su	suricata	snmpd	apache	sshd	su	suricata	snmpd	apache
OpenChat	19	7	24	3	5	35.562 (0.165)	12.607 (0.019)	68.383 (0.049)	4.991 (0.029)	11.633 (0.006)
Mistral	19	7	–	13	4	47.269 (0.43)	17.475 (0.007)	–	25.587 (0.073)	7.556 (0.01)
Wizardlm2	194	182	–	16	78	1809.316 (0.476)	1309.777 (0.161)	–	92.519 (1.236)	323.523 (0.085)

Table 4 Performance of template detection methods (log message batch size 10)

Method	Number of correctly detected templates (number of all detected templates in parentheses)					F1-score				
	sshd	su	suricata	snmpd	apache	sshd	su	suricata	snmpd	apache
OpenChat	25 (31)	10 (13)	3 (15)	7 (7)	5 (7)	0.746	0.769	0.286	1	0.714
Mistral	17 (25)	10 (10)	–	0 (1)	0 (2)	0.557	0.87	–	0	0
Wizardlm2	9 (51)	1 (9)	–	5 (6)	3 (7)	0.207	0.091	–	0.769	0.429
Drain	29 (44)	9 (16)	3 (80)	6 (8)	2 (4)	0.725	0.621	0.07	0.8	0.364

Best results highlighted in bold

bilities of different algorithms to infer ground truth templates from insufficient event log data. In Table 5, relevant results are displayed.

According to Table 5, the performance of all methods deteriorated, and the decline in the F1-score was most notable for Drain. For example, for the *sshd* data set, the F1-score decreased by 0.5, with the number of correctly detected templates decreasing from 29 to 9. Also, in the case of *suricata* and *apache* data sets, the number of correctly identified templates (and thus the F1-score) dropped to 0. Drain achieved its highest F1-score of 0.533 on the *snmpd* data set.

For LLM-based methods, the decline in the performance of OpenChat was less severe, with its performance not deteriorating on *su* and *suricata* data sets. Also, OpenChat achieved the highest F1-score on four data sets. Mistral managed to reach the best F1-score of 0.783 on the *su* data set, with other F1-scores remaining below 0.4. As for Wizardlm2, its F1-score dropped from 0.429 to 0 on the *apache* data set, whereas performance degraded less for other data sets.

These results illustrate that even if there is not enough information in log messages for inferring ground truth templates, some LLMs like OpenChat are able to achieve noticeably better results than traditional algorithms like Drain. To investigate that phenomenon, we analyzed detected templates and relevant log data manually, and Fig. 4 displays two example cases to illustrate the following discussion.

In the case of the first scenario, OpenChat and Mistral were able to infer the ground truth template from insufficient event log data, whereas Drain produced a template that is

```
# event log data (two messages in sshd data set)

sshd[17250]: Failed none for invalid user edward from
192.168.0.89 port 44460 ssh2
sshd[17266]: Failed none for invalid user edward from
192.168.0.89 port 44632 ssh2

# template detected by Drain
<*> Failed none for invalid user edward from
192.168.0.89 port <*> ssh2

# ground truth template (detected by OpenChat and Mistral)
sshd[<*>]: Failed <*> for invalid user <*> from
<*> port <*> ssh2

# event log data (15 messages in sshd data set)

sshd[11356]: Protocol major versions differ for 192.168.3.141:
SSH-2.0-OpenSSH_5.3 vs. SSH-9.9-OpenSSH_5.0
sshd[11357]: Protocol major versions differ for 192.168.3.141:
SSH-2.0-OpenSSH_5.3 vs. SSH-1.33-OpenSSH_5.0
...

# template detected by Drain
<*> Protocol major versions differ for 192.168.3.141:
SSH-2.0-OpenSSH_5.3 vs. <*>

# template detected by OpenChat
sshd[<*>]: Protocol major versions differ for <*>:
SSH-<*>-OpenSSH_<*> vs. SSH-<*>-OpenSSH_<*>

# ground truth template
sshd[<*>]: Protocol major versions differ for <*>: <*> vs. <*>
```

Fig. 4 Inferring ground truth templates from insufficient log data

not very informative (most variable fields are not properly identified). That example highlights one major advantage of LLM-based approaches over traditional algorithms, and we discovered a number of similar cases during our analysis, where templates detected by LLM-based approaches from insufficient log data were more informative. The superiority

Table 5 Performance of template detection methods without P1 (log message batch size 10)

Method	Number of correctly detected templates					F1-score				
	sshd	su	suricata	snmpd	apache	sshd	su	suricata	snmpd	apache
OpenChat	17	10	3	5	3	0.507	0.769	0.286	0.714	0.429
Mistral	11	9	–	0	0	0.361	0.783	–	0	0
Wizardlm2	7	1	–	4	0	0.161	0.091	–	0.615	0
Drain	9	5	0	4	0	0.225	0.345	0	0.533	0

Best results highlighted in bold

of LLM-based approaches over traditional algorithms comes from the fact that LLMs are trained on a very large set of widely different texts, allowing LLM-based methods to identify patterns that are not evident from the analyzed event log data.

The second scenario in Fig. 4 illustrates the fact that although LLM-based approaches are not always able to produce a result that is identical to a ground truth template, the result can provide new insights into event log data by identifying regularities inside variable log message words. For example, the template by OpenChat in Fig. 4 not only indicates that two SSH protocol fields are variables but also reveals that protocol strings of all messages begin with the prefix *SSH-* and contain the substring *-OpenSSH_*. In contrast, traditional algorithms treat log message words as atoms and are not able to discover character patterns they might contain.

That advantage of LLM-based approaches over traditional algorithms is essential if ground truth templates have words that contain *(*)* as a substring, and the template mining algorithm is expected to identify them exactly as they appear in ground truth templates. For example, about 89% of ground truth templates of data sets from Table 1 have such a nature, and without using the P2 principle for assessing Drain's performance, the performance would decrease significantly. Also, without using neither P1 nor P2 principle, the performance of OpenChat, Mistral, and Wizardlm2 remained the same as reported in Table 5, because we applied P2 only to Drain during the experiments. On the other hand, disabling both principles reduced the F1-score for Drain to 0 on all data sets, illustrating the strengths of LLM-based approaches over traditional algorithms when complex ground truth templates need to be identified.

4.3.3 Analysis of incorrectly detected templates

As pointed out in a recent paper by Khan et al. [25], existing works have not investigated incorrectly detected templates enough, and studying them helps to understand which types of detection errors are most common for specific algorithms. If t is a template, let $\log(t)$ denote the set of all potential messages (i.e., not only the messages in the evaluation data set!) which are matching template t . If $\log(t_1) \subset \log(t_2)$,

```
# a ground truth template
sshd[<*>]: Server listening on <*> port <*>.

# an over-generalized (OG) template
sshd[<*>]: <*>

# two under-generalized (UG) templates
sshd[<*>]: Server listening on 0.0.0.0 port <*>.
sshd[<*>]: Server listening on :: port <*>.

# a mixed (MX) template
<*> Server listening on 0.0.0.0 port <*>
```

Fig. 5 Types of incorrectly detected templates

then template t_1 is called *more specific* than template t_2 (i.e., any message matched by t_1 is also matched by t_2 , but not vice versa). Also, t_2 is called *more general* than t_1 . According to [25], an incorrect template is *over-generalized* (OG) if it is more general than some ground truth template, and is *under-generalized* (UG) if it is more specific than some ground truth template. If an incorrect template is neither OG nor UG, it is called *mixed* (MX).

Figure 5 provides an example message for each type. For example, the last example from Fig. 5 is of type MX since, unlike the ground truth template, it assumes the constant value 0.0.0.0 for the *interface IP address* field. On the other hand, unlike the ground truth template, it does not assume that the message begins with the program name *sshd* and ends with the dot character. Table 6 provides data about the types of incorrect templates from our evaluation.

As seen in Table 6, LLM-based approaches tend to produce very few MX templates, and for OpenChat no such templates were generated on five data sets. In contrast, for Drain, the MX templates formed a significant part among incorrectly detected templates on *sshd*, *su*, and *suricata* data sets. As discussed above, traditional algorithms (including Drain) are not able to identify character patterns inside message words, and our investigation revealed that many incorrect templates from Drain represent the same scenario as illustrated by the MX template from Fig. 5. However, on the *snmpd* and *apache* data sets, Drain produced just two incorrectly detected templates, and no prominent pattern of MX templates emerged.

When manually inspecting OG templates, we discovered that Mistral generated an OG template *snmpd[<*>]: <*>* on

Table 6 Incorrectly detected templates by type (log message batch size 10)

Method	sshd			su			suricata			snmpd			apache		
	OG	UG	MX	OG	UG	MX	OG	UG	MX	OG	UG	MX	OG	UG	MX
OpenChat	2	4	0	2	1	0	0	12	0	0	0	0	2	0	0
Mistral	1	5	2	0	0	0	–	–	–	1	0	0	2	0	0
Wizardlm2	0	42	0	0	8	0	–	–	–	0	0	1	0	4	0
Drain	1	1	13	1	2	4	0	43	34	0	2	0	2	0	0

the *snmpd* data set during earlier phases of the event log processing. Because detected templates are used for filtering out log messages for performance optimization reasons (see line 4 in Algorithm 1 in Sect. 3), no further messages were submitted to Mistral for analysis after the detection of this template, although this would have allowed to find additional templates from the remaining event log data. As a result, that led Mistral to identify only one template *snmpd*[*<*>*]: [*<*>*]. On the *apache* data set, Mistral suffered from the same issue, and the following two OG templates which matched all event log messages prevented the proper analysis of most event log data:

apache2: PHP Notice: <> in <*> on line <*>*

apache2: PHP Warning: <> in <*> on line <*>*

Also, as discussed in Sect. 4.3.1, the incorrect exclusion of log messages from processing by OG templates can have side effects on runtime, and allowed Mistral and Wizardlm2 to process the *suricata* data set with the shortest execution times for $k = 2$.

Since other LLM-based template detection approaches such as LILAC [22] and Lunar [23] utilize similar work principles of employing already detected templates as filters, our finding illustrates one design limitation of current LLM-based approaches. Namely, if an underlying LLM is prone to produce overly general templates, a mechanism must be implemented to identify and drop such templates. Unfortunately, whereas some overly general templates like the OG template from Fig. 5 are fairly easy to detect, there are other more subtle cases, and their identification is a non-trivial task. For example, aforementioned two OG templates detected by Mistral on the *apache* data set contain six words and three [*<*>*], giving them a look of a regular template and hiding their too general nature.

When investigating UG templates, we found that some of them provide novel insights into the event log data despite being incorrect according to ground truth. The most notable case involved the *suricata* data set, which was the most challenging for all algorithms because Mistral and Wizardlm2 failed to process it, whereas the highest F1-score remained below 0.3 and was achieved by OpenChat (see Table 4). In the case of OpenChat, all incorrectly identified templates were of type UG and have been shown in Fig. 6. The total number of UG templates was 12, and they were more specific than

```
# template for Suricata IDS alerts from 153 ET signatures
suricata[<*>]: [<*>] ET <*> [Classification: <*>]
[Priority: <*>] {<*>} <*> -> <*>:<*>

# template for Suricata IDS alerts from 30 ETPRO signatures
suricata[<*>]: [<*>] ETPRO <*> [Classification: <*>]
[Priority: <*>] {<*>} <*> -> <*>:<*>

# templates for Suricata IDS alerts from 10 GPL signatures

suricata[<*>]: [<*>] GPL DNS named authors attempt
[Classification: Attempted Information Leak]
[Priority: <*>] {UDP} <*> -> <*>:<*>

suricata[<*>]: [<*>] GPL FTP MKD overflow
[Classification: Attempted Administrator Privilege Gain]
[Priority: <*>] {TCP} <*> -> <*>:<*>
...
```

Fig. 6 Templates from OpenChat describing Suricata IDS alerts

the ground truth template shown in Fig. 1 (Sect. 1), which covered over 99% of the *suricata* data set.

Also, in Suricata IDS alert messages (see Fig. 1 in Sect. 1), the first word of the alert string identifies the signature supplier. For example, in two alerts from Fig. 1 the supplier identifiers are *ET* (free signatures from EmergingThreats) and *GPL* (community signatures). Another identifier present in the *suricata* data set is *ETPRO* (commercial signatures from EmergingThreats).

Despite being incorrectly identified, the UG templates from Fig. 6 reveal the fact that IDS alerts have been produced by signatures from three suppliers. Since for *ET* and *ETPRO*, a larger number of signatures produced many different alert strings, alerts for both suppliers have been summarized by a single template. However, since for the *GPL* supplier, only 10 signatures triggered alerts, individual templates have been created for each *GPL* signature, providing more details for the human analyst. Since OpenChat did not generate any other incorrectly identified templates, templates from Fig. 6 can be regarded as a concise and meaningful summary of Suricata IDS alerts, providing useful insights not present in ground truth.

As for Drain (the other algorithm that managed to process the *suricata* data set), it neither detected the ground truth template for IDS alerts nor produced a meaningful summary for them. When investigating 77 incorrectly detected templates from Drain, we found that the issue was related to the common design limitation of traditional algorithms, which

```
# an example message from HTTP event log

GET: Client 10.4.2.6 requested URL www.example.com/test.html
with protocol HTTP/1.1 (referer http://www.example.com)
and received 5132 bytes with status code 200 (user agent
Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36)

# templates summarizing HTTP event log by HTTP method

GET: Client <*> requested URL <*> with protocol HTTP/<*>
(referer <*>) and received <*> bytes with status code <*>
(user agent <*>)

POST: Client <*> requested URL <*> with protocol HTTP/<*>
(referer <*>) and received <*> bytes with status code <*>
(user agent <*>)

PUT: Client <*> requested URL <*> with protocol HTTP/<*>
(referer <*>) and received <*> bytes with status code <*>
(user agent <*>)
...
```

Fig. 7 Templates from OpenChat describing HTTP requests

assume that the log messages matching the same template contain the same number of words. Since Drain has this particular design limitation, it could not detect meaningful templates from Suricata IDS alert data.

Finally, to estimate the capabilities of LLM-TD to process large non-*syslog* data sets, we applied it to a textual event log of all HTTP requests observed in a large organizational network. The event log contained 15,311,709 messages describing HTTP requests from 36,821 clients, and all messages were following the same format. Figure 7 displays an example anonymized message from this event log (since the log data had a confidential nature, we were not able to release this data set publicly).

When applying LLM-TD with OpenChat to this event log 10 times, it was processed within 6–7 min despite its large size of 3.9 GB, producing identical output for all 10 iterations. Furthermore, the memory consumption of LLM-TD remained modest, ranging from 65.5 to 67.4 MB across all iterations (we observed similar memory footprints lower than 70 MB for all other experiments described earlier in this paper).

Although OpenChat was not able to detect a single template for all messages, it was able to meaningfully summarize them with 11 templates (see Fig. 7), which represented distinct HTTP methods identified in the event log (for one HTTP method, two templates were created which reflected different usage patterns of this method). However, 17 log messages remained uncovered by templates from Fig. 7 and were reported to the end user. When inspecting them more closely, we discovered that six messages represented previously unknown malicious HTTP requests with binary bytes used in the HTTP method field.

Scenarios from Figs. 6, 7, and 4 demonstrate that LLM-based approaches like OpenChat have a potential not only for template detection but also for *discovering previously unknown knowledge* that is not represented by ground truth.

Also, it highlights the fact that detected templates deserve qualitative analysis during evaluations to better assess the knowledge discovery capabilities of different algorithms.

5 Discussion

In this section, we will summarize the main findings of the evaluation presented in Sect. 4.

First, as discussed in Sect. 4.2, existing performance metrics are not entirely suited for a meaningful comparison of LLM-based and traditional algorithms since the latter have several design limitations. Also, adjusting the ground truth templates according to these limitations (e.g., as proposed in [25]) would not allow us to evaluate the unique capabilities of LLM-based approaches. As a solution, we have proposed two heuristic principles for assessing the correctness of detected templates.

Second, our results show that local LLMs with parameter size 7B can feature a good performance when used without fine-tuning, employing the in-context learning paradigm in an unsupervised fashion. In particular, the LLM-TD algorithm with the OpenChat LLM featured the best performance among all evaluated methods, including Drain, a highly capable traditional algorithm [24]. That allows security analysts to employ local LLMs for unsupervised event log analysis without the need to create labeled data sets. Moreover, since security event logs contain highly sensitive data, they can be analyzed without submitting sensitive information to external service providers.

Whereas existing LLM-based algorithms [19–23] have attempted to detect one template during each LLM query, the LLM-TD algorithm presented in this paper utilizes LLMs for extracting several templates at once during every query. As the results from Sect. 4.3 indicate, such a detection strategy is a viable alternative to existing approaches, and it allows to reduce computationally expensive interaction with LLMs.

One drawback of LLM-based template detection algorithms is their need for more computational resources than traditional algorithms require, which leads to longer execution times. For example, during our evaluations, a high-end GPU was used by local LLMs (see Sect. 4.1), whereas traditional algorithms do not need such dedicated hardware. In addition, Drain featured significantly faster execution times than LLM-TD with OpenChat, Mistral, and Wizardlm2 (see Table 2 in Sect. 4.3.1). Also, in the case of one challenging data set (*suricata*), Mistral and Wizardlm2 were not able to process it even within 10 h for most log message batch sizes.

Furthermore, LLM-based approaches have several unique capabilities that traditional algorithms lack. According to our findings (e.g., see Fig. 4 in Sect. 4.3.2), LLMs can infer informative templates even from insufficient event log data. Also,

LLMs are able to detect character patterns inside log message words which can yield novel insights into event log data.

Our study has also identified several design limitations of LLM-based and traditional algorithms, which can interfere with the template detection process. According to recent research literature [20, 26], many traditional algorithms incorrectly assume that log messages matching the same template have the same number of words, and our study has confirmed the negative impact of this assumption. Also, since traditional algorithms consider the log message words as atoms, detected templates are often less informative. As for the limitations of the LLM-based algorithms, they commonly employ already detected templates to exclude log messages from further analysis. However, if overly general templates are identified during the early phases of event log processing, a large fraction of relevant messages could be left unprocessed.

Finally, as our study has indicated, qualitative analysis of detected templates is important since templates that deviate from ground truth can provide novel insights into event log data. Unfortunately, existing studies have not paid enough attention to this aspect. However, as our study has shown, LLM-based algorithms are able to find previously unknown knowledge from event log data, and this warrants the qualitative analysis of *all* detected templates.

6 Conclusion and future work

In the current paper, we have presented the LLM-TD algorithm for unsupervised template detection from security event logs, and have demonstrated that small local LLMs can be successfully used for template discovery purposes. Furthermore, our study has pointed out the drawbacks of existing performance metrics, and has proposed several heuristic principles for addressing these drawbacks. The current paper has also highlighted the advantages of LLM-based template detection over traditional algorithms, and has emphasized the importance of qualitative analysis of all identified templates. The following paragraphs will outline some research directions for future work.

In previous studies, a generic wildcard notation (e.g., $\langle * \rangle$) has been used for denoting a variable part in log messages that match the template, and the current paper has adopted the same approach. However, such variable parts can have a different nature. For example, in Fig. 1 (Sect. 1), $\langle * \rangle$ can match IP addresses, port and process numbers, user names, etc. Instead of using a generic $\langle * \rangle$ notation, more specific notations for different data types (e.g., $\langle * : \text{number} \rangle$ or $\langle * : \text{ipaddress} \rangle$) would make detected templates significantly more informative. Whereas traditional algorithms would need regular expression-based event log preprocessing to achieve that goal (e.g., see [5, 12]), LLMs have the

potential of detecting such templates from the original event log data, and studying approaches of finding these templates is a valuable further research direction.

In addition to detecting more informative templates, LLMs could be utilized to discover regular expressions that correspond to these templates since human analysts often employ them instead of templates for actual event log processing tasks. However, according to our preliminary experiments, discovering regular expressions directly from event log messages using LLMs is a complex task. To address that issue, LLMs could be tasked with detecting some simplified regular-expression-like intermediate formats from log messages so that these intermediate formats could be converted into regular expressions with a little effort.

One of the limitations of the current study is the template detection performance analysis for a limited number of local LLMs. Since new LLMs are constantly released, evaluating the template detection capabilities of a larger number of local LLMs is an open future research topic.

Finally, as mentioned in Sect. 1, there is a shortage of publicly available recent security event logs, which would facilitate the cyber security-centric event log analysis research. Although this paper has published several such event logs, creating additional realistic security event log data sets and releasing them into the public domain remains an important task for future work.

Acknowledgements This work was supported by the Estonian Centre of Excellence in Artificial Intelligence (EXAI), funded by the Estonian Ministry of Education and Research grant TK213.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Splunk (2024). <https://www.splunk.com/>. Accessed 10 Jul
2. ElasticStack (2024). <https://www.elastic.co/elastic-stack>. Accessed 10 Jul
3. Vaarandi, R., Blumbergs, B., Çalışkan, E.: Simple event correlator—best practices for creating scalable configurations. In: IEEE CogSIMA Conference, pp. 96–100 (2015). <https://doi.org/10.1109/COGSIMA.2015.7108181>

4. RFC3164 (2024). <https://www.ietf.org/rfc/rfc3164.txt>. Accessed 10 Jul
5. He, P., Zhu, J., Zheng, Z., Lyu, M.R.: Drain: an online log parsing approach with fixed depth tree. In: IEEE International Conference on Web Services, pp. 33–40 (2017). <https://doi.org/10.1109/ICWS.2017.13>
6. Tang, L., Li, T., Perng, C.-S.: LogSig: generating system events from raw textual logs. In: ACM International Conference on Information and Knowledge Management, pp. 785–794 (2011). <https://doi.org/10.1145/2063576.2063690>
7. Makanju, A.A.O., Zincir-Heywood, A.N., Milios, E.E.: Clustering event logs using iterative partitioning. In: ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1255–1264 (2009). <https://doi.org/10.1145/1557019.1557154>
8. Du, M., Li, F.: Spell: streaming parsing of system event logs. In: IEEE International Conference on Data Mining, pp. 859–864 (2016). <https://doi.org/10.1109/ICDM.2016.0103>
9. Jiang, Z.M., Hassan, A.E., Flora, P., Hamann, G.: Abstracting execution logs to execution events for enterprise applications. In: International Conference on Quality Software, pp. 181–186 (2008). <https://doi.org/10.1109/QSIC.2008.50>
10. Fu, Q., Lou, J.-G., Wang, Y., Li, J.: Execution anomaly detection in distributed systems through unstructured log analysis. In: IEEE International Conference on Data Mining, pp. 149–158 (2009). <https://doi.org/10.1109/ICDM.2009.60>
11. Shima, K.: Length matters: clustering system log messages using length of words (2016). <https://arxiv.org/abs/1611.03213>
12. Vaarandi, R., Pihelgas, M.: LogCluster—a data clustering and pattern mining algorithm for event logs. In: International Conference on Network and Service Management, pp. 1–7 (2015). <https://doi.org/10.1109/CNSM.2015.7367331>
13. Vaarandi, R.: A data clustering algorithm for mining patterns from event logs. In: IEEE Workshop on IP Operations and Management, pp. 119–126 (2003). <https://doi.org/10.1109/IPOM.2003.1251233>
14. Nagappan, M., Vouk, M.A.: Abstracting log lines to log event types for mining software system logs. In: IEEE Working Conference on Mining Software Repositories, pp. 114–117 (2010). <https://doi.org/10.1109/MSR.2010.5463281>
15. Hamooni, H., Debnath, B., Xu, J., Zhang, H., Jiang, G., Mueen, A.: LogMine: fast pattern recognition for log analytics. In: ACM International Conference on Information and Knowledge Management, pp. 1573–1582 (2016). <https://doi.org/10.1145/2983323.2983358>
16. Messaoudi, S., Panichella, A., Bianculli, D., Briand, L., Sasnauskas, R.: A search-based approach for accurate identification of log message formats. In: Conference on Program Comprehension, pp. 167–177 (2018). <https://doi.org/10.1145/3196321.3196340>
17. Mizutani, M.: Incremental mining of system log format. In: IEEE International Conference on Services Computing, pp. 595–602 (2013). <https://doi.org/10.1109/SCC.2013.73>
18. Dai, H., Li, H., Chen, C.-S., Shang, W., Chen, T.-H.: Logram: efficient log parsing using n-gram dictionaries. IEEE Trans. Softw. Eng. **48**(3), 879–892 (2020). <https://doi.org/10.1109/TSE.2020.3007554>
19. Ma, Z., Chen, A.R., Kim, D.J., Chen, T.-H., Wang, S.: LLMParser: an exploratory study on using large language models for log parsing. In: International Conference on Software Engineering, Article 99 (2024). <https://doi.org/10.1145/3597503.3639150>
20. Xu, J., Yang, R., Huo, Y., Zhang, C., He, P.: DivLog: log parsing with prompt enhanced in-context learning. In: International Conference on Software Engineering, Article 199 (2024). <https://doi.org/10.1145/3597503.3639155>
21. Le, V.-H., Zhang, H.: Log parsing with prompt-based few-shot learning. In: International Conference on Software Engineering, pp. 2438–2449 (2023). <https://doi.org/10.1109/ICSE48619.2023.00204>
22. Jiang, Z., Liu, J., Chen, Z., Li, Y., Huang, J., Huo, Y., He, P., Gu, J., Lyu, M.R.: LILAC: log parsing using LLMs with adaptive parsing cache. In: International Conference on the Foundations of Software Engineering, pp. 137–160 (2024). <https://doi.org/10.1145/3643733>
23. Huang, J., Jiang, Z., Chen, Z., Lyu, M.R.: LUNAR: unsupervised LLM-based log parsing (2024). <https://arxiv.org/abs/2406.07174v2>
24. Zhu, J., He, S., Liu, J., He, P., Xie, Q., Zheng, Z., Lyu, M.R.: Tools and benchmarks for automated log parsing. In: International Conference on Software Engineering: Software Engineering in Practice, pp. 121–130 (2019). <https://doi.org/10.1109/ICSE-SEIP.2019.00021>
25. Khan, Z.A., Shin, D., Bianculli, D., Briand, L.: Guidelines for assessing the accuracy of log message template identification techniques. In: International Conference on Software Engineering, pp. 1095–1106 (2022). <https://doi.org/10.1145/3510003.3510101>
26. Gasimov, O., Vaarandi, R., Pihelgas, M.: Comparative analysis of pattern mining algorithms for event logs. In: IEEE International Conference on Cyber Security and Resilience, pp. 1–7 (2023). <https://doi.org/10.1109/CSR57506.2023.10224996>
27. Zhu, J., He, S., He, P., Liu, J., Lyu, M.R.: Loghub: a large collection of system log datasets for AI-driven log analytics. In: International Symposium on Software Reliability Engineering, pp. 355–366 (2023). <https://doi.org/10.1109/ISSRE59848.2023.00071>
28. Wang, G., Cheng, S., Zhan, X., Li, X., Song, S., Liu, Y.: OpenChat: advancing open-source language models with mixed-quality data. In: International Conference on Learning Representations (2024). <https://arxiv.org/pdf/2309.11235>
29. Mistral (2024). <https://mistral.ai/news/announcing-mistral-7b/>. Accessed 10 Jul
30. Wizardlm2 (2024). <https://wizardlm.github.io/WizardLM2/>. Accessed 10 Jul

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.